

SplitSync: Bank Group-Level Split-Synchronization for High-Performance DRAM PIM

Byungkuk Yoon, Sanghyeok Han, Gyeonghwan Park, and Jae-Joon Kim
Seoul National University, Republic of Korea
[bkyoon, hans778, gyeonghwan.park, kimjaejeon]@snu.ac.kr

Abstract—Processing in Memory (PIM) architectures enhance memory bandwidth by utilizing bank-level parallelism, typically implemented with a SIMD structure where all banks operate simultaneously under a single command. However, this synchronous approach requires the activation of all banks before computation, leading to activation times that exceed computation times, limiting performance gain. Recently, asynchronous execution PIM has been proposed as an alternative, allowing banks to operate asynchronously and overlap activation with processing to hide the row activation overhead. While effective at reducing row activation overhead, the independent operation requires large shared accumulators for each bank group, increasing area overhead. To address the issues, we propose bank group (BG)-level split synchronization DRAM PIM, where each bank group operates asynchronously to hide row activation overhead while operating synchronously within the bank group to eliminate the need for shared accumulators. Evaluation results show that our proposed design achieves an average throughput improvement of 1.70x and 1.06x compared to conventional PIM and asynchronous execution PIM. Furthermore, the area overhead per processing unit (PU) increases by only 1.5% compared to conventional PIM and is significantly lower than that of asynchronous execution PIM.

I. INTRODUCTION

DRAM Processing-in-memory (PIM) has emerged as a promising approach to accelerate memory-intensive workloads by harnessing bank-level parallelism (BLP) [20, 4, 9, 18, 2, 19]. This technique leverages the inherent parallelism of DRAM banks to enhance memory bandwidth, making it well-suited for tasks like neural network acceleration, particularly General Matrix-Vector Multiplication (GEMV) [20, 13, 14]. Despite its potential, the practical performance of DRAM PIM often falls short due to the overhead associated with memory operations, such as row activation/precharge, input writing, and result reading. A critical bottleneck in achieving full BLP in DRAM PIM is the t_{FAW} (Four Activate Window) constraint, which limits the number of banks that can be activated within a specified time frame. This limitation is designed to prevent voltage drops within DRAM but introduces substantial row activation overhead when activating multiple banks, negatively impacting overall throughput.

Recent approaches have sought to mitigate this constraint. For example, DRAM PIMs in [14, 11, 12] leverage circuit-level innovations by incorporating reservoir capacitors to enable simultaneous activation of 16 banks, effectively bypassing the t_{FAW} limitation. Let us call this DRAM PIM style as ACT16-PIM in this paper. While this solution improves performance, it comes at the cost of significant area overhead due to the additional reservoir capacitors. Alternatively, asynchronous execution PIM architecture [8] activates and processes rows across banks in a staggered manner, overlapping activation and computation to reduce overhead. However, this method complicates input data distribution, requiring large shared accumulators for gathering and accumulating partial results across banks, leading to increased area and design complexity.

To address these limitations, we propose SplitSync PIM, a DRAM-PIM architecture that combines the benefits of synchronous and asynchronous operations. Within each bank group, synchronous operations enable efficient input broadcasting and computation without

the need for shared accumulators, thereby minimizing area overhead. Meanwhile, asynchronous operation between bank groups allows activation and computation phases to overlap, significantly reducing row activation overhead while maintaining compliance with the t_{FAW} constraint. This design also eliminates the need for large reservoir capacitors and reduces area complexity compared to ACT16-PIM. Additionally, we introduce modifications to the I/O architecture that allow independent operations across bank groups, mitigating I/O conflicts and enabling simultaneous execution of specific operation pairs: input writing with input broadcasting, input broadcasting with another input broadcasting, and input broadcasting with result reading.

Our main contributions are as follows:

- We propose a SplitSync DRAM PIM architecture that performs synchronous computations within bank groups and asynchronous operations between bank groups. The synchronous operations facilitate implementation with minimal area overhead, while asynchronous operations between bank groups effectively reduce row activation overhead.
- To resolve I/O conflicts during asynchronous operations, we introduce new I/O architecture to support independent operations between bank groups, thereby improving overall performance.
- We compare SplitSync with previous DRAM PIM architectures using several neural network benchmarks and demonstrate that it provides higher throughput than previous works by a large margin under the same area budget.

II. BACKGROUND

A. Operation and Architecture of Conventional DRAM PIM

Conventional DRAM PIM [15, 13, 14, 11, 4, 20] operations extend memory bandwidth by exploiting Bank-Level Parallelism (BLP), which accelerates memory-intensive workloads. To maximize bank-level parallelism, all DRAM banks operate concurrently within a SIMD (Single Instruction, Multiple Data) architecture. As shown in Fig. 1a, in order to access all banks simultaneously, all banks are activated before the PIM operation begins. After activation, the columns of the activated rows are accessed sequentially to perform the computation. Since the host controls the DRAM through DRAM commands, new DRAM commands need to be introduced to manage PIM operations and ensure smooth execution.

The main focus of PIM is on General Matrix-Vector Multiplication (GEMV) operations, a key component of neural network computation. GEMV is a memory-bound operation, where a large weight matrix, typically not reused, must be loaded from memory to process a relatively small input vector. Fig. 1b illustrates the tiling method used for GEMV computation in conventional PIM. The rows of the weight matrix are mapped to DRAM banks, tiled according to DRAM row size, and then stored in the corresponding banks and rows. The input vector is also tiled to match the DRAM row size and written into the global buffer (GB) in preparation for computation. GEMV performs MAC (Multiply-Accumulate) operations, sequentially multiplying the weight tiles by the input vector for each bank and accumulating

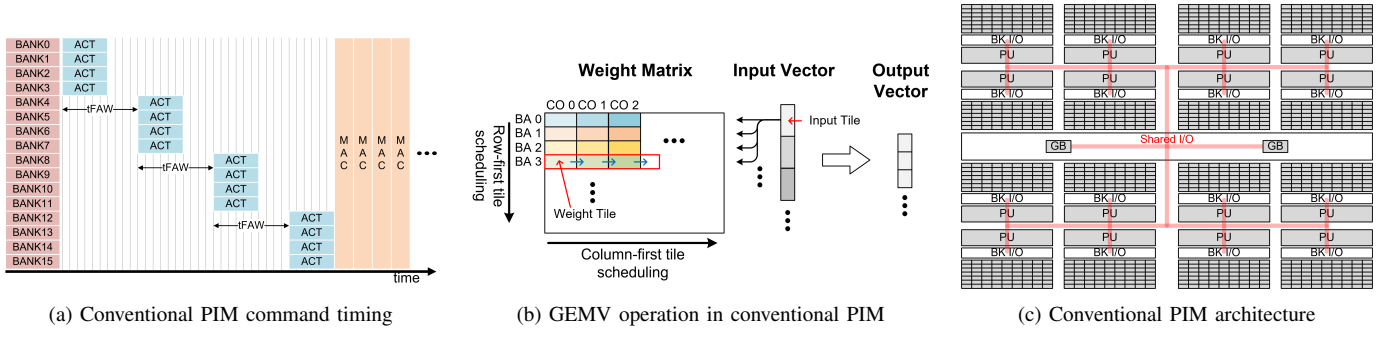


Fig. 1: Conventional DRAM PIM architecture and its operation

the results in the result latch. Once the computation for one tile is finished, the process repeats for the next tile (corresponding to the next DRAM row) until all computations are completed.

The tile scheduling for GEMV computation can be classified into row-first or column-first tile scheduling, depending on how the tiles are selected during computation. In column-first tile scheduling, the tile moves to the next matrix column along the matrix row, and after completing operations on one matrix row, it moves to the next matrix row. The advantage of this scheduling is that partial sums can be continuously accumulated in the result latch to compute the final sum. However, since the input is used only within each tile, frequent input writes are required, which increases power consumption and introduces writing overhead, degrading performance. In contrast, row-first tile scheduling moves the tile along the weight columns, enabling reuse of the input vector. In this case, either new result latches are required for each tile, or the partial sums must be read out by the host. Although partial sums must be read out after each tile operation, row-first scheduling offers performance and power advantages because the partial sums are typically much smaller than the input tile. This smaller data size allows the host to handle aggregation while the PIM continues processing the next tile.

Fig. 1c shows the architecture of a conventional DRAM PIM. To perform the GEMV operation, processing units (PUs) are placed adjacent to each bank. These PUs receive weights from their respective banks and inputs from the global buffer to execute MAC operations. Since the input vector is shared across all PUs, it is broadcast via shared DRAM I/O, while the weights are delivered through local I/O from each bank. The global buffer, implemented in SRAM, is positioned on either side of the peripheral circuit area, typically sized to match the DRAM row. The results accumulated in the PUs are read out through the DRAM read path via shared I/O. Since the result of each PU is smaller than the column data width, the results from multiple PUs are aggregated and output together.

B. Challenges of DRAM PIM Operation

While DRAM PIM can boost memory bandwidth through bank-level parallelism, it often falls short of expected performance levels in practice due to significant overheads, such as input writes, row activation/precharge, and result reads required for PIM operations. In GEMV operations, where activated DRAM rows are not reused, the repeated all-bank activation and precharge for each tile make row activation/precharge overhead particularly critical.

The time required for all-bank activation is determined by two timing parameters, t_{RRD} and t_{FAW} . t_{RRD} represents the minimum time interval between activation commands, while t_{FAW} defines the minimum timing window that allows up to four activation commands. Let t_{ACT} denote the time required for all-bank activation, which can be expressed in terms of the aforementioned timing parameters:

TABLE I: Timing parameters and memory configuration for row activation overhead calculation

	Parameter	Value
Timing Parameters	t_{RRD}	9
	t_{FAW}	32
	t_{CCD}	2
	t_{RCD}	24
	t_{RP}	24
Memory Configuration	N	16
	Columns	64

$$t_{ACT} = \max(t_{FAW}, t_{RRD}) \times \left(\left\lceil \frac{N}{4} \right\rceil - 1 \right) + t_{RCD}$$

Here, t_{RCD} represents the minimum time interval between activation and read commands, and N denotes the number of banks in PIM. When the MAC operation follows read timing, the time for a single tile MAC operation, denoted as t_{MAC} , can be expressed as:

$$t_{MAC} = t_{CCD} \times (\text{columns} - 1)$$

where t_{CCD} is the minimum time interval between two read or write commands, and columns is the number of columns in a DRAM row.

In a single tile operation, the row activation/precharge overhead can be represented as:

$$\frac{t_{ACT} + t_{RP}}{t_{ACT} + t_{MAC} + t_{RP}}$$

where t_{RP} refers to the precharge time. Based on the timing parameters in Table I, this overhead is found to be 52.9%, indicating that more time is spent on activation/precharge than on computation.

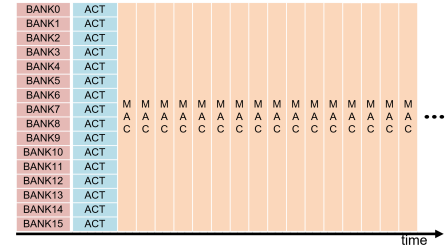
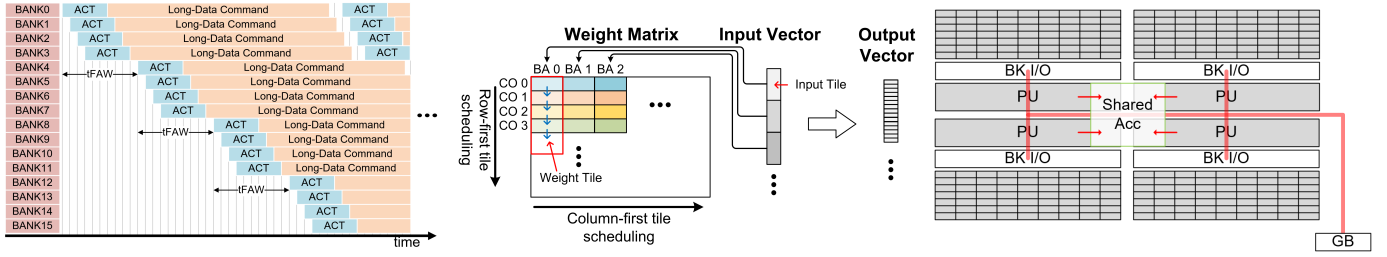


Fig. 2: ACT16-enabled DRAM PIM command timing

C. ACT16-enabled DRAM PIM

A DRAM PIM architecture has been introduced to reduce row activation overhead by eliminating the t_{FAW} constraint at the circuit level, enabling the simultaneous activation of all 16 banks (ACT16) (Fig. 2) [14, 11]. DRAM operates using various internal voltages, and certain timing parameters, such as t_{FAW} , are constrained to prevent failures due to internal voltage drops. In this design, additional reservoir capacitors are placed near the processing units to mitigate internal voltage drops caused by the simultaneous activation of all



(a) Asynchronous execution PIM command timing (b) GEMV operation in asynchronous execution PIM (c) Asynchronous execution PIM architecture

Fig. 3: Asynchronous execution DRAM PIM architecture and its operation [8]

16 banks. As a result, the ACT16 approach enables PIM operation without the t_{FAW} constraint but it incurs significant area overhead. While the exact area of the reservoir capacitors has not been provided, we estimated their area from the die photo, and estimate that the area overhead per PU is approximately 20–25%. The GEMV operation and overall architecture in this PIM design remain unchanged from those in conventional DRAM PIM.

D. Asynchronous Execution PIM

Recently, asynchronous execution PIM architecture has been proposed as another solution to reduce row activation overhead [8]. In this architecture, as shown in Fig. 3a, each bank operates asynchronously, reducing row activation overhead by overlapping row activation and operations across different banks. Since a single command controls only one bank, a long-data command is proposed to avoid command conflict. This command automatically processes operations across an entire row and is issued once at the start of the computation to each banks.

Fig. 3b illustrates the GEMV operation in this architecture. Unlike conventional PIM, small blocks of the weight matrix row are mapped to individual banks, arranged along the matrix columns. Since each bank operates at different timings, broadcasting input data simultaneously to all banks is not feasible. Instead, each bank receives and processes its designated data block independently, reusing it across the entire DRAM row with the long-data command. As operations progress across columns, accumulation cannot occur within individual banks but should take place between banks. To address this, a shared accumulator is placed at the center of each bank group, as shown in Fig. 3c. The shared accumulator receives data asynchronously transmitted from each bank, identifies the values to be added based on the address information, and performs the accumulation.

In this architecture, the operation of a single row in a single bank takes $t_{RCD} + t_{MAC} + t_{RP}$, and t_{ACT} is given by:

$$t_{ACT} = \max(t_{FAW}, 4 \times t_{RRD}) \times \left(\left\lceil \frac{N}{4} \right\rceil - 1 \right) + [(N - 1) \bmod 4] \times t_{RRD}$$

Thus, the total execution time for M tiles is $(t_{RCD} + t_{MAC} + t_{RP}) \times M + t_{ACT}$, and the impact of t_{RRD} and t_{FAW} diminishes as M increases.

Despite demonstrating good performance, this architecture has several implementation challenges.

First, the size of the shared accumulator scales with the DRAM row size. According to [8], the shared accumulator incurs an area overhead of approximately 10% per PU based on a 1KB row size. For memory technologies such as GDDR6 [7], which feature row sizes of 2KB or larger, and DDR5 [3]/LPDDR5 [17], which can also support 2KB row size, the area overhead increases to 15-20%.

Second, positioning the shared accumulator centrally within a bank group presents additional challenges. Banks within the bank group

share address/data buses and various control circuits. This shared infrastructure requires that bus distribution, data gathering circuits, and control circuits already occupy most of the central space in the bank group. Consequently, integrating a shared accumulator in this central area creates competition for space, making it difficult to prioritize its placement over essential DRAM operation blocks.

Third, asynchronous operations introduce potential conflicts in shared I/O during data transfers. Specifically, conflicts can occur when loading input blocks from the global buffer to each bank and when reading computation results from the shared accumulators. After a computation is completed and banks prepare to move to the next tile, those that have finished must wait for others to complete. This results in idle periods until all banks synchronize and results are read out, allowing the next tile to load. Conversely, synchronous operations ensure that all banks handle input and output simultaneously, eliminating these conflicts. As a result, asynchronous operations can introduce significant overhead, particularly with row-first tile scheduling, though column-first tile scheduling can help mitigate this issue.

III. SPLITSYNC DRAM PIM DESIGN

A. Bank Group-Level Split Synchronization

As discussed in the previous section, conventional DRAM PIM experiences significant performance degradation due to row activation overhead. In contrast, both ACT16-enabled PIM and asynchronous execution PIM reduce this overhead but at the cost of increased area requirements. Specifically, ACT16-enabled PIM needs substantial space for reservoir capacitors, while asynchronous execution PIM demands a large shared accumulator. To overcome the performance and area challenges, we propose a bank group-level split synchronization architecture that effectively reduces row activation overhead with minimal area overhead. This architecture operates synchronously within a bank group and asynchronously between bank groups, as illustrated in Fig. 4a.

The synchronous operation within a bank group provides two main advantages. First, since each bank group consists of four banks, all banks within the group can be activated concurrently without being constrained by t_{FAW} . Second, synchronous operations within a bank group enable the global buffer to broadcast input data across the banks. This capability allows the rows of the weight matrix to be mapped to banks in a manner similar to conventional PIM, facilitating intra-bank accumulation and removing the need for a shared accumulator. Moreover, the asynchronous operation between bank groups allows for overlapping row activations and operations across different bank groups, which helps to further reduce overhead, as previously mentioned.

B. SplitSync DRAM PIM Architecture

The SplitSync PIM architecture is shown in Fig. 4c. There are three major changes from conventional PIM designs. First, the I/O

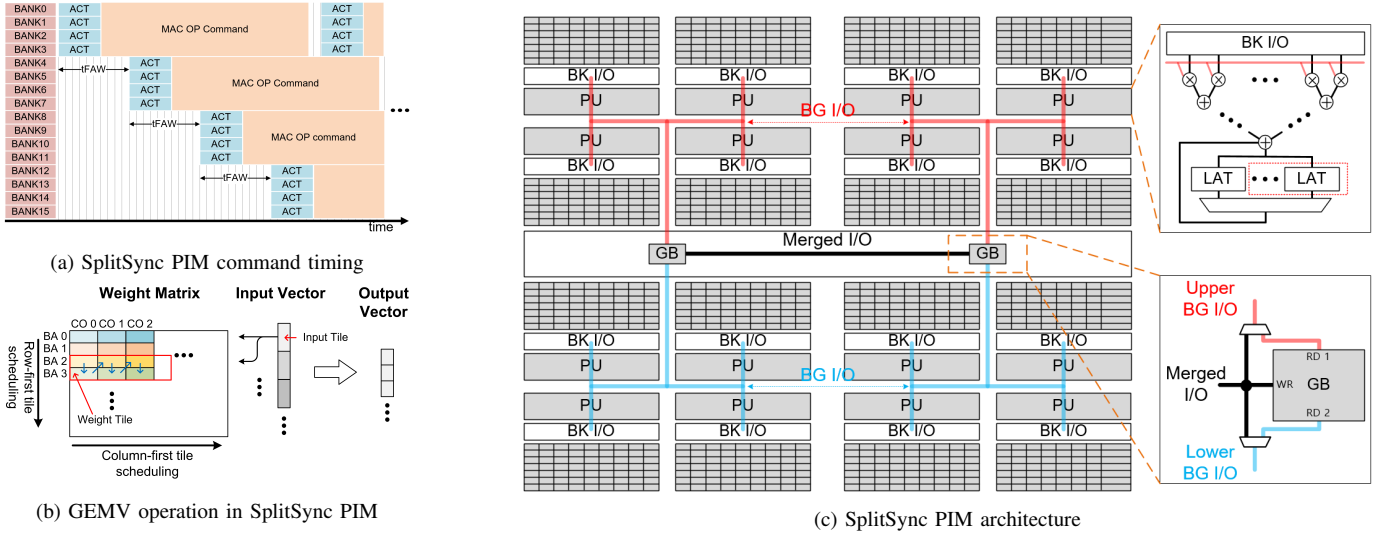


Fig. 4: Proposed SplitSync DRAM PIM architecture and its operation

architecture is updated from a shared I/O to a combination of bank-group I/O (BG I/O) and merged I/O. Second, the global buffer is now connected to the BG I/O, enabling input broadcasting through the BG I/O. Third, the number of result latches per PU is increased from 1 set to 4 sets, with each set consisting of 16 latches in BF16 format. Here, 1 set refers to 1 unit.

Resolving the I/O conflict Due to the different operation timings between bank groups, input broadcasting is not possible on shared I/O. To address the issue, we introduce bank group-specific I/O, called BG I/O (red and cyan lines in Fig. 4c), which is shared within each BG and supports data movement between bank groups, allowing different inputs to be broadcast to each bank group. The BG I/O is connected to the global buffer and facilitates localized broadcasting, but it does not connect directly to the DRAM interface; instead, it links through the merged I/O. The merged I/O is positioned in the peripheral circuit area and aggregates the BG I/Os (black line in Fig. 4c), enabling seamless access to banks during read and write operations. This architecture ensures that read and write operations at the bank group level do not disrupt PIM operations in other bank groups. As a result, as shown in Fig. 5, these updates to the I/O architecture enables simultaneous operations such as input writing with input broadcasting, input broadcasting with another input broadcasting, and input broadcasting with result reading.

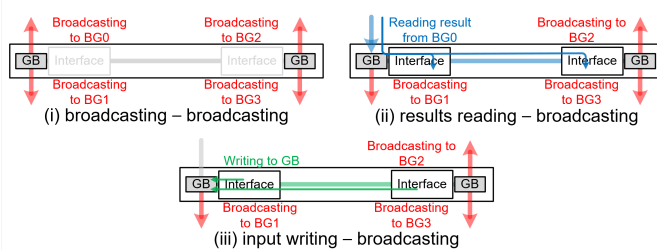


Fig. 5: Simultaneous operations in SplitSync DRAM PIM

GEMV operation in SplitSync PIM The method for tiling GEMV in SplitSync PIM is illustrated in Fig. 4b. While the GEMV tiling approach is fundamentally similar to that of conventional PIM, two key distinctions arise due to the architectural changes in SplitSync PIM.

First, the weight matrix is divided into half and mapped across the left and right bank groups, with the input vector split accordingly between the left and right GBs. This is necessary because the GB is horizontally divided, and each bank group is connected to only one of the GBs via BG I/O (Fig. 4c). Since the connections between GBs uses merged I/O, broadcasting input data from the opposite-side GB that is not connected via BG I/O can cause inefficiencies and potential I/O conflicts. To avoid these issues, the input vector can be either duplicated across both GBs or partitioned between them, with partitioning generally being the more efficient solution.

Second, more than two weight matrix rows are mapped to a single DRAM row to address the mismatch between DRAM row size and GB size. In conventional PIM architectures, even when the GB is split, it functions as a unified large GB during computation. However, in SplitSync PIM, the split GBs operate independently, reducing the GB size available to each bank group. This creates a mismatch between DRAM row size and GB size. To manage this, more result latches are required. Nonetheless, this approach is still more efficient than increasing the GB size. This mapping approach also reduces the frequency of input broadcasting, improving power efficiency. Additionally, increasing the number of result latches enables multiple matrix rows to be stored consecutively within a single DRAM row. This is particularly useful for operations involving numerous small-dimension GEMV operations, such as those found in the multi-head attention mechanism of transformer decoders [22]. In these cases, if there are insufficient result latches, the operation must pause during computation to perform intermediate readouts, which can degrade performance. In SplitSync PIM, four sets of latches are assigned to each PU to allow efficient readout at the BG level. With each PU outputting four units in BF16 format, and four banks per bank group, the total output is 256 bits per bank group, aligning with the column width. The column width is the amount of data transferred in a single read operation.

MAC operation command Since SplitSync PIM enables independent operation of each bank group, command conflicts must be considered. These conflicts can be managed in a manner similar to the long-data command used in asynchronous execution PIM. However, unlike the long-data command, the MAC operation command in SplitSync PIM also convey information about how many matrix rows (2, 4, or 8) are stored in each DRAM row. The MAC

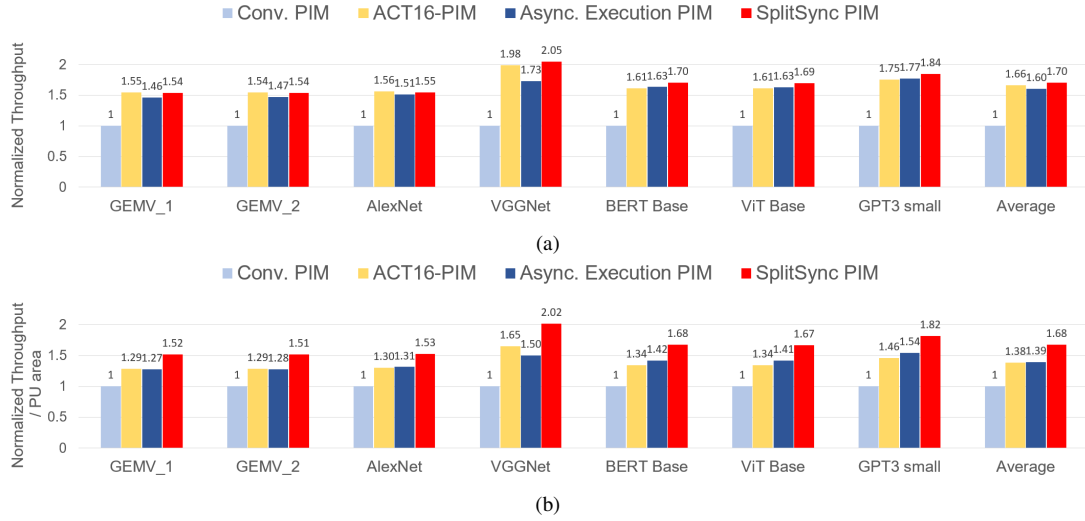


Fig. 6: Throughput comparison across benchmarks. (a) Normalized to conventional DRAM PIM (b) Normalized throughput per PU area.

operation command allows accessing either the entire row or half, which is crucial for improving resource utilization when dealing with matrices with small dimensions or tiled fragments. Once a bank group is activated, the MAC operation command is issued, enabling sequentially access to columns within the row and initiating computations.

IV. EVALUATION

A. Methodology

Seven microbenchmarks are used to evaluate the performance of four PIM architectures (Table II). Specifically, two microbenchmarks are assigned to each of the GEMV and CNN models [10, 21], and three to Transformer models [5, 6, 1], focusing solely on matrix operations that PIM can handle in each model. For the CNN model, convolution operations are converted into General Matrix-Matrix Multiplication (GEMM), which is then executed through repeated GEMV operations.

The performance evaluation is conducted by modifying the cycle-level DRAMsim3 [16]. All simulations are based on the same GDDR6 configuration (Table II bottom). Since the performance of each PIM can vary depending on tile scheduling, the most optimal scheduling method is applied for each PIM: Conventional PIM, ACT16-enabled PIM and SplitSync PIM use row-first tile scheduling, while Asynchronous Execution PIM uses column-first tile scheduling.

TABLE II: Benchmark and GDDR6 PIM Configuration

Benchmark	Target Layer
GEMV1	2K × 2K GEMV
GEMV2	4K × 4K GEMV
AlexNet	5 CONV, 3 FC
VGGNet	13 CONV, 3 FC
BERT Base	12 TRANS
ViT Base	12 TRANS
GPT-3 small	12 TRANS
GDDR6 PIM Configuration	
Bank groups	4 bank groups per channel
Banks	4 banks per bank group
Column width	256 bits
Row Size	2 KB
Density	4 Gb
Clock frequency	1.5 GHz
Global buffer	2 KB

B. Performance

Fig. 6a shows a throughput comparison across seven microbenchmarks, with all results normalized to the throughput of conventional PIM. The three PIM architectures designed to reduce row activation overhead consistently outperform conventional PIM in all cases. Although the extent of performance improvement varies depending on the benchmark, SplitSync PIM shows the highest performance on average. The performance improvement is more pronounced in neural network models than GEMV benchmarks due to varying matrix dimensions within the models. The two GEMV benchmarks have matrix dimensions that align perfectly with the banks and rows during tiling, resulting in efficient use of DRAM resources. In contrast, neural network models often have matrix dimensions that do not map perfectly to DRAM rows or banks, leading to partial utilization of activated rows and higher row activation overhead. Additionally, when the matrix column size is considerably smaller than the DRAM row size, multiple matrix rows can be computed within a single DRAM row, giving an additional edge to both asynchronous execution PIM and SplitSync PIM. Overall, across the seven benchmarks, SplitSync PIM achieves an average throughput improvement of 1.70x, 1.02x, and 1.06x compared to conventional PIM, ACT16-enabled PIM, and asynchronous execution PIM, respectively. A even more important point is that SplitSync achieves higher performance compared to other designs while maintaining the smallest area overhead relative to conventional PIM. The next section provides a detailed analysis of the area overhead.

C. Area Overhead

We estimated the area overhead of SplitSync PIM using a 28nm CMOS process. While the size of the global buffer does not increase compared to conventional PIM, adding a read port results in a 62% increase in its area. However, since the global buffer accounts for less than 1% of the total PIM area, this increase is negligible. Increasing the number of result latches to 4 sets per processing unit incurs a 1.5% area overhead per PU, which is significantly smaller than the area overhead observed in ACT16-enabled PIM and asynchronous execution PIM. As described in Section II-C and II-D, ACT16-enabled PIM incurs 20-25% and asynchronous execution PIM incurs 15-20% area overhead. Fig. 6b shows the throughput results normalized by PU area for the seven benchmarks.

Although proposed architectures use different approaches, all three designs effectively address the t_{FAW} constraint, resulting in similar performance. However, the area used to resolve the t_{FAW} constraint varies across architectures, revealing notable performance differences in iso-area performance. Factoring in PU area, SplitSync PIM shows performance improvements of 1.67x, 1.21x, and 1.20x compared to conventional PIM, ACT16-enabled PIM, and asynchronous execution PIM, respectively.

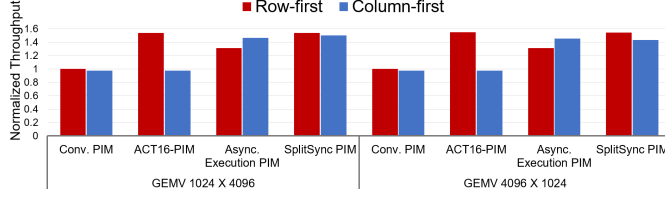


Fig. 7: Impact of tile scheduling on PIM performance

D. Impact of Tile Scheduling

Fig. 7 shows the results of how tile scheduling affects PIM performance. We evaluate two cases of GEMV: $1K \times 4K$ and $4K \times 1K$, with results normalized to the row-first tile scheduling of conventional PIM. For conventional PIM, row-first tile scheduling shows slightly better results than column-first scheduling, but the difference is minimal, with performance remaining nearly identical. In contrast, the other architectures show significant performance differences depending on the tile scheduling methods, most notably in ACT16-enabled PIM. This disparity occurs because, as row activation overhead is reduced, other overheads that were previously masked become more visible.

In asynchronous execution PIM, the performance difference is primarily due to frequent result reading in row-first tile scheduling. Unlike other architectures, asynchronous execution PIM cannot hide result reads behind precharge-activation due to I/O conflict issues and the high volume of results to be read out. As a result, it shows worse performance with row-first tile scheduling. With column-first tile scheduling, performance degrades due to the increased number of input writes, particularly affecting ACT16-enabled PIM. In conventional PIM, input writes are masked by the long activation time, but in ACT16-enabled PIM, shorter activation time prevents this masking, resulting in a greater performance impact. Conversely, SplitSync PIM and asynchronous execution PIM mitigate this issue by enabling more input reuse through additional result latches. Furthermore, in asynchronous execution PIM, only small blocks are written per tile, allowing frequent writes to be hidden during activation.

E. Impact of t_{FAW} and Row Size

Row activation overhead is determined by the row activation time and the operation time. Row activation time is primarily influenced by t_{FAW} , while operation time depends largely on t_{CCD} and the row size. Since t_{CCD} is the inverse of the operation frequency, it can be excluded from this analysis, leaving changes in t_{FAW} and row size as the key factors determining row activation overhead. While there is a positive correlation between t_{FAW} and row size, the exact relationship varies by design. Therefore, we treat these as independent variables and analyze their impacts separately. Once the relationship between the two variables is established during the design phase, the analysis can focus on the relevant regions of the results. Fig. 8 presents these findings as a heatmap, with throughput normalized to the overall average. In conventional architectures, throughput generally decreases as row activation overhead increases

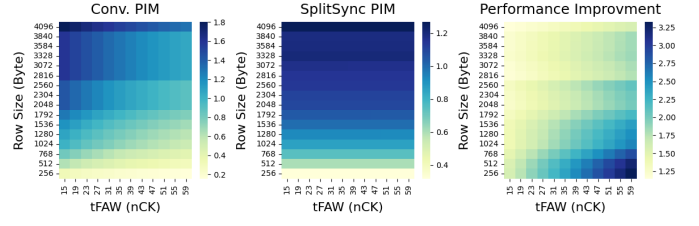


Fig. 8: Performance impact when varying t_{FAW} and row size

towards the lower right of the heatmap. In contrast, SplitSync PIM is less affected by t_{FAW} , with row size playing a more important role in its throughput. SplitSync PIM consistently shows a performance improvement of at least 1.15x across all areas. As expected, the performance gain becomes larger when row activation overhead becomes greater (moving towards the lower right of the heatmap).

F. Energy

We use the DRAMsim3 simulator to evaluate the energy consumption during PIM operations. For a $4K \times 4K$ GEMV, ACT16-enabled PIM, asynchronous execution PIM and SplitSync PIM consume 97%, 89% and 86% of the energy compared to conventional PIM, respectively. ACT16-enabled PIM exhibits slight energy reduction over conventional PIM since energy savings come solely from reduced stand-by energy as a result of shorter processing times. In asynchronous execution PIM, input data is loaded only once per row, reducing I/O energy consumption. SplitSync PIM achieves further energy savings through shorter BG I/O paths and lower broadcasting frequency, as multiple rows are processed together, decreasing overall I/O energy usage. In both SplitSync PIM and asynchronous execution PIM, reduced I/O energy consumption and lower stand-by energy due to shorter processing times enhance energy efficiency.

V. CONCLUSION

In this paper, we present SplitSync PIM, a DRAM PIM architecture designed to reduce row activation overhead through bank group-level split synchronization. This approach allows each bank group to operate asynchronously to hide row activation overhead while maintaining synchronous operations within each bank group to enable input broadcasting at the bank group-level. SplitSync PIM addresses the t_{FAW} constraint without relying on the large reservoir capacitors and drivers used in ACT16-enabled PIM, and it eliminates the need for a shared accumulator in asynchronous execution PIM. Additionally, I/O conflicts occurring from asynchronous operations are resolved through a modified I/O architecture, enabling all bank groups to operate independently. Consequently, SplitSync PIM effectively reduces row activation overhead efficiently, with minimal area overhead.

VI. ACKNOWLEDGEMENT

This work was supported in part by Institute of Information communications Technology Planning Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2023-00208606, NeuroHub+: Scheduler and Simulator for General In-Memory Neural Network Accelerators, RS-2022-II220266, Development of Ultra-Low Power Low-Bit Precision Mixed-mode SRAM PIM, IITP-2023-RS-2023-00256081: artificial intelligence semiconductor support program to nurture the best talents), BK21 FOUR program at Seoul National University, and ISRC at Seoul National University. The EDA tool was supported by the IC Design Education Center (IDEC). (Corresponding Author: Jae-Joon Kim).

REFERENCES

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al. "Language models are few-shot learners". In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS '20. Vancouver, BC, Canada: Curran Associates Inc., 2020.
- [2] S. Cho, H. Choi, E. Park, H. Shin, and S. Yoo. "McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge". In: *IEEE Access* 8 (2020), pp. 135223–135243.
- [3] *DDR5 SDRAM Standard*. JESD79-5. JEDEC Solid State Technology Association. 2020.
- [4] F. Devaux. "The true processing in memory accelerator". In: *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society. 2019, pp. 1–24.
- [5] J. Devlin. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *arXiv preprint arXiv:1810.04805* (2018).
- [6] A. Dosovitskiy. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).
- [7] *Graphics Double Data Rate (GDDR6) SGRAM Standard*. JESD250D. JEDEC Solid State Technology Association. 2018.
- [8] H. Kal, C. Yoo, and W. W. Ro. "AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory". In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO '23. Toronto, ON, Canada: Association for Computing Machinery, 2023, pp. 815–827.
- [9] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, et al. "GradPIM: A Practical Processing-in-DRAM Architecture for Gradient Descent". In: *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 2021, pp. 249–262.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).
- [11] D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G.-M. Hong, et al. "A 1nm 1.25V 8Gb 16Gb/s/Pin GDDR6-Based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep Learning Application". In: *IEEE Journal of Solid-State Circuits* 58.1 (2023), pp. 291–302.
- [12] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, et al. "System architecture and software stack for GDDR6-AiM". In: *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE. 2022, pp. 1–25.
- [13] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, et al. "25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications". In: *2021 IEEE International Solid-State Circuits Conference (ISSCC)*. Vol. 64. 2021, pp. 350–352.
- [14] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, et al. "A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications". In: *2022 IEEE International Solid-State Circuits Conference (ISSCC)*. Vol. 65. 2022, pp. 1–3.
- [15] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, et al. "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product". In: *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 2021, pp. 43–56.
- [16] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob. "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator". In: *IEEE Computer Architecture Letters* 19.2 (2020), pp. 106–109.
- [17] *Low Power Double Data Rate (LPDDR) 5/5X*. JESD209-5C. JEDEC Solid State Technology Association. 2023.
- [18] Y. Paik, C. H. Kim, W. J. Lee, and S. W. Kim. "Achieving the Performance of All-Bank In-DRAM PIM With Standard Memory Interface: Memory-Computation Decoupling". In: *IEEE Access* 10 (2022), pp. 93256–93272.
- [19] J. Park, S. Lee, and J. Lee. "NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on pim". In: *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE. 2023, pp. 1–6.
- [20] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo. "McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37.11 (2018), pp. 2613–2622.
- [21] K. Simonyan and A. Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).
- [22] A. Vaswani. "Attention is all you need". In: *Advances in Neural Information Processing Systems* (2017).